

PRODUCT REQUIREMENTS DOCUMENT (PRD) – VERSION 3.0



AI-Powered Event-Driven Ticket Analysis and Processing Platform on AWS

Version: 3.0

Project Type: Major MCA Cloud Computing Project

Domain: Cloud Engineering, DevOps, AI Integration

Architecture Style: Cloud-Native, Event-Driven, Scalable Microservice-Oriented Design

1. Executive Summary

CloudQueueX AI is a cloud-native support ticket processing platform designed to automate ticket submission, AI-assisted analysis, prioritization, routing, storage, notification, and monitoring using AWS managed services.

The system combines:

- Event-Driven Architecture
- AI-Based Ticket Analysis
- Queue-Based Processing
- Infrastructure as Code
- Cloud Automation
- Monitoring & Observability

to create a scalable and highly available support management platform.

2. Problem Statement

Organizations receive hundreds of support requests daily.

Traditional ticket handling faces several challenges:

Current Problems

- ✗ Manual ticket categorization
- ✗ Incorrect priority assignment
- ✗ Delayed response times

- ✗ Ticket processing bottlenecks
- ✗ Lack of automation
- ✗ Poor scalability
- ✗ High operational cost
- ✗ Limited monitoring and visibility

These problems impact productivity, service quality, and customer satisfaction.

3. Project Goal

To build an intelligent cloud-native platform capable of:

- Receiving support tickets
 - Analyzing tickets using AI
 - Assigning category and priority
 - Processing tickets asynchronously
 - Storing ticket information
 - Sending notifications automatically
 - Monitoring platform health
 - Automating infrastructure deployment
-

4. Project Vision

To demonstrate how modern organizations can leverage AWS Cloud Services, AI, and Event-Driven Architecture to build scalable enterprise support systems.

5. Business Objectives

Primary Objectives

- ✓ Automate ticket classification
- ✓ Improve response time
- ✓ Reduce manual effort
- ✓ Build scalable architecture

- ✓ Demonstrate Infrastructure as Code
 - ✓ Implement cloud-native design
 - ✓ Improve operational efficiency
 - ✓ Enable AI-assisted ticket management
-

6. Target Audience

End Users

- Customers
- Employees
- Students
- Support Requestors

Administrative Users

- Support Engineers
 - Cloud Engineers
 - DevOps Engineers
 - System Administrators
 - Operations Teams
-

7. System Scope

The platform will:

- Accept support tickets
- Analyze tickets using AI
- Categorize requests
- Assign priority levels
- Route tickets
- Queue requests
- Process requests asynchronously
- Store ticket information

- Send notifications
- Monitor application activity
- Automate infrastructure deployment

8. High-Level Architecture

Users

↓

Application Load Balancer (ALB)

↓

EC2 Flask Web Portal

↓

Amazon API Gateway

↓

Amazon Bedrock AI Analysis Engine

↓

Amazon SQS Queue

↓

Lambda Worker / Python Worker

↓

Amazon DynamoDB

↓

Amazon SNS

↓

Support Team Notification

↓

Amazon CloudWatch

(Monitoring & Logging)



AWS CloudFormation

(Infrastructure Automation)

9. Project Modules

Module 1 – User Ticket Management

Description

Provides interface for users to create support requests.

Features

- ✓ Ticket Submission
- ✓ Ticket ID Generation
- ✓ Ticket Status Tracking
- ✓ Ticket Search
- ✓ Ticket History

Input Fields

- User Name
- Email Address
- Category
- Subject
- Description

Output

TKT-2026-1001

Unique Ticket Identifier

Module 2 – Amazon Bedrock AI Analysis Engine

Description

Uses Generative AI to analyze submitted tickets.

AWS Service

Amazon Bedrock

Features

- ✓ Ticket Categorization
- ✓ Priority Detection
- ✓ Intelligent Summarization
- ✓ Team Recommendation
- ✓ Context Understanding

Example

Input:

Production Linux server is down and users cannot access the application.

Output:

Category: Infrastructure

Priority: Critical

Summary:

Production server outage affecting users.

Assigned Team:

Linux Operations

Module 3 – API Management Layer

AWS Service

Amazon API Gateway

Features

- ✓ API Management
 - ✓ Request Validation
 - ✓ Secure API Access
 - ✓ Backend Integration
 - ✓ Scalable API Layer
-

Benefits

- Secure communication
 - Enterprise-grade API architecture
 - Easy integration with serverless services
-

Module 4 – Event-Driven Processing Engine

AWS Service

Amazon SQS

Features

- ✓ Queue-Based Processing
 - ✓ Asynchronous Architecture
 - ✓ Message Durability
 - ✓ High Throughput
 - ✓ Decoupled Components
-

Workflow

Ticket Created

↓

SQS Queue

↓

Worker Processing

Module 5 – Ticket Lifecycle Management

Features

- ✓ Ticket Validation
- ✓ Ticket Processing
- ✓ Status Updates
- ✓ Ticket History

Status Flow

Submitted

↓

Analyzing

↓

Queued

↓

Processing

↓

Resolved

↓

Closed

Module 6 – Database Management Module

AWS Service

Amazon DynamoDB

Features

- ✓ Ticket Storage
 - ✓ Status Storage
 - ✓ Metadata Storage
 - ✓ Fast Retrieval
 - ✓ Scalable NoSQL Database
-

Data Stored

- Ticket ID
 - Name
 - Email
 - Category
 - Priority
 - Summary
 - Status
 - Timestamp
-

Module 7 – Notification & Alerting Module

AWS Service

Amazon SNS

Features

- ✓ Email Notifications
 - ✓ Ticket Alerts
 - ✓ Status Updates
 - ✓ Automated Communication
-

Notification Events

- Ticket Created

- Ticket Assigned
 - Ticket Resolved
 - Ticket Closed
-

Module 8 – Monitoring & Observability Module

AWS Service

Amazon CloudWatch

Features

- ✓ Application Monitoring
 - ✓ Queue Monitoring
 - ✓ Error Tracking
 - ✓ Performance Metrics
 - ✓ Log Aggregation
-

Metrics

- API Requests
 - Queue Depth
 - Processing Time
 - Error Count
 - Worker Health
-

Module 9 – Infrastructure Automation Module

AWS Service

CloudFormation

Features

- ✓ Infrastructure as Code
- ✓ Automated Provisioning

- ✓ Repeatable Deployment
 - ✓ Version Controlled Infrastructure
-

Resources Created

- EC2
 - IAM
 - Security Groups
 - API Gateway
 - SQS
 - DynamoDB
 - SNS
 - CloudWatch
-

Module 10 – High Availability & Scalability Module

AWS Services

- Application Load Balancer
- Auto Scaling
- DynamoDB

Features

- ✓ Load Balancing
 - ✓ Traffic Distribution
 - ✓ Horizontal Scaling
 - ✓ High Availability
-

Benefits

System remains responsive during high traffic.

Module 11 – Security & Access Control Module

AWS Services

- IAM
- Security Groups

Features

- ✓ Least Privilege Access
 - ✓ Role-Based Permissions
 - ✓ Secure Resource Access
 - ✓ Traffic Filtering
-

10. Functional Requirements

FR-01

User submits ticket.

FR-02

AI analyzes ticket.

FR-03

Priority assigned automatically.

FR-04

Ticket pushed to SQS.

FR-05

Worker processes ticket.

FR-06

Ticket stored in DynamoDB.

FR-07

Notification sent using SNS.

FR-08

Monitoring data generated.

FR-09

Infrastructure deployed via CloudFormation.

11. Non-Functional Requirements

Availability

Target:

99.9%

Scalability

System should scale automatically.

Reliability

Queue-based fault-tolerant processing.

Security

IAM Roles and Security Groups.

Maintainability

CloudFormation-based deployments.

12. AWS Services Used

Service	Purpose
EC2	Web Portal Hosting
ALB	Load Balancing
API Gateway	API Layer
Amazon Bedrock AI Analysis	
SQS	Event Queue
DynamoDB	Ticket Database

Service	Purpose
SNS	Notifications
CloudWatch	Monitoring
IAM	Security
Security Groups	Network Security
CloudFormation	Infrastructure as Code

Total AWS Services Used

11 AWS Services

13. Technology Stack

Frontend

- HTML
- CSS
- JavaScript
- Flask

Backend

- Python

Cloud

- AWS

AI

- Amazon Bedrock

Infrastructure

- CloudFormation YAML

Version Control

- Git
 - GitHub
-

14. Logging Strategy

Example:

[INFO] Ticket Submitted

[INFO] Ticket ID: TKT-1001

[INFO] Category: Infrastructure

[INFO] Priority: Critical

[INFO] Stored in DynamoDB

[INFO] SNS Notification Sent

15. Cost Optimization Strategy

- ✓ AWS Free Tier
 - ✓ CloudFormation Automation
 - ✓ DynamoDB On-Demand
 - ✓ SQS Pay-As-You-Go
 - ✓ Delete Resources After Testing
-

16. Future Enhancements (Version 4.0)

- AI Chatbot Support
- Bedrock Agents
- CI/CD Pipeline
- GitHub Actions
- Docker Containers

- Kubernetes (EKS)
 - Route53 Domain
 - Multi-Region Disaster Recovery
 - Predictive Ticket Analytics
 - Automated Resolution Suggestions
-

17. Success Criteria

Project is successful if:

- ✓ Ticket submitted successfully
 - ✓ AI analyzes ticket
 - ✓ Priority assigned automatically
 - ✓ Ticket enters SQS queue
 - ✓ Worker processes request
 - ✓ Data stored in DynamoDB
 - ✓ Notification sent via SNS
 - ✓ Monitoring data generated
 - ✓ Infrastructure deployed using CloudFormation
-

Final Project Statement

CloudQueueX AI is an AI-powered, cloud-native, event-driven ticket analysis and processing platform built on AWS using Amazon Bedrock, API Gateway, SQS, DynamoDB, SNS, CloudWatch, CloudFormation, EC2, and ALB. The platform automates ticket classification, prioritization, routing, storage, notification, monitoring, and infrastructure deployment while demonstrating modern cloud engineering, DevOps practices, AI integration, and scalable enterprise architecture.